

[← Go Back](#)

Hardware

Alvik

Tutorials

[Arduino Alvik API Overview](#)[Getting Started with Alvik](#)[Start With Alvik And Block-Based Language](#)[Setting up Alvik on Arduino IDE](#)[Alvik User Manual](#)[Home](#) / [Hardware](#) / [Alvik](#) / **Alvik User Manual**

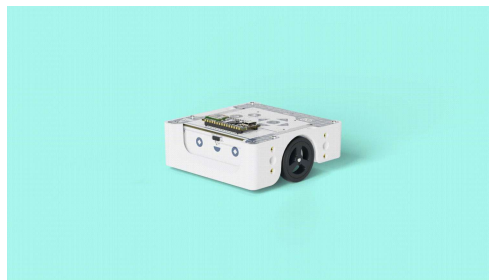
Alvik User Manual

Discover how to use and program Arduino® Alvik's robot

Author · Paolo Cavagnolo, Pedro Miguel, Jose Garcia · Last revision · 10/16/2025

Introduction

Arduino® Alvik is a powerful and versatile robot specifically designed for programming and STEAM education.



Powered by the Arduino® Nano ESP32, Alvik offers diverse learning paths through different programming languages including MicroPython, Arduino C, and block-based coding; enabling different possibilities to explore Robotics, IoT, and Artificial Intelligence.

In this tutorial, you will find useful information to get started, test, and maintain Alvik.

What Is Alvik

Alvik is a robot with two controllers and a wide range of sensors and actuators. The main controller is the Arduino Nano ESP32 attached to the top of the robot while there is an STM32 controller integrated into the robot that takes care of the low-level commands such as reading the sensors and

ON THIS PAGE

Introduction

- [What Is Alvik](#) +
- [Alvik's Brain](#) +
- [Alvik's Body](#) +
- [Alvik's Sensors](#) +
- [Alvik's Actuators](#) +
- [Out Of The Box Modes](#)
- [Alvik's Firmware](#) +
- [Deep Dive Programming Alvik](#) +
- [Talking With Other Devices!](#) +
- [Maintenance](#) +
- [Expanding the Robot](#) +
- [Want More?](#)
- [Need Help](#)

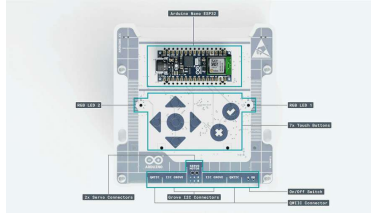
Abstract

18650 Li-Ion battery attached to the

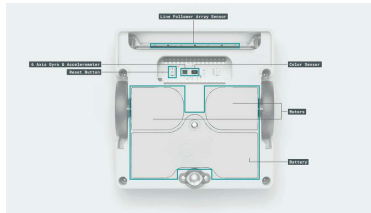
USB-C® to USB-C® cable



 If you remove the hardware from the



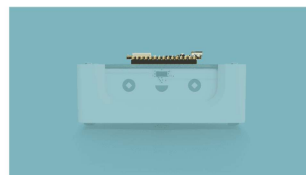
Alvik top components



Alvik bottom components

Alvik's Brain

The **Nano ESP32** is the brain of the Alvik robot, acting as the central hub that runs the logic and makes key decisions. While it plays a critical role in processing high-level commands and overseeing the robot's operations, it doesn't directly control the motors or sensors. Instead, the Nano ESP32 communicates with the **STM32** (the body), sending instructions for actions like movement, sensor readings, or status updates. This collaboration allows the robot to perform tasks and routines based on the data it receives, enabling it to make informed decisions as we've programmed it to be.



Alvik Brain

Nano ESP32

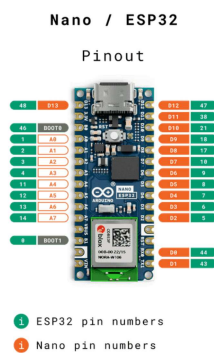
The **Nano ESP32** is the board used to

flash memory, and a Wi-Fi® enabled chip packed into a tiny circuit board.



You can find out more about this board in the [Nano ESP32 documentation](#).

Please note that when using MicroPython the pin number reflects the GPIO on the ESP32-S3, not the Nano board. Use the **green labeled number** in the following image. You can read more about this [here](#).



Nano ESP32 pinout

Alvik's Body

Using the same analogy, the body of the Alvik robot includes everything else from the sensors and motors to the connectors supporting external modules and, of course, the STM32, which acts like the nervous system of the robot. Just as the human nervous system transmits signals between the brain and various parts of the body, the STM32 transmits commands and collects data from the robot's components. When the Nano ESP32, acting as the brain, sends commands, the STM32 executes these low-level operations, such as moving the motors, flashing the LEDs, or interacting with external devices. The STM32 is essential for carrying out the detailed actions based on the high-level instructions from the Nano ESP32, ensuring that the robot's movements and functions are performed accurately.



Alvik Body

Controller - STM32

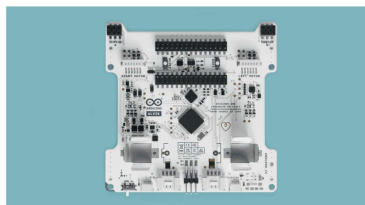
The main core of the robot is the STM32 ARM Cortex-M4 32 Bit controller. It handles the low-level commands such as reading sensors and moving motors. You can access it through a set of dedicated APIs from the Nano ESP32.

You can learn more about the available functions for Alvik in the following [Alvik's API Documentation](#).



Although Alvik's "out of the box" experience is designed in MicroPython, there are C++ APIs that allow you to program Alvik in the Arduino IDE, these APIs are 1 to 1 compatible with the MicroPython APIs, which means that the [Alvik's API Documentation](#) will give you information for both MicroPython and C++ programming experiences.

The latest firmware of the STM32 can be found at [this link](#), and [here](#) is the guide to flash it.

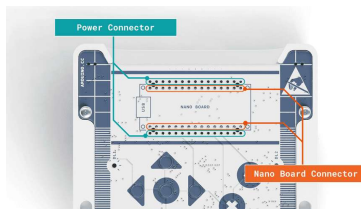


Alvik STM32

Header Connectors

On the top face of the Alvik, you can find 2 pairs of connectors. Both have 15 holes in

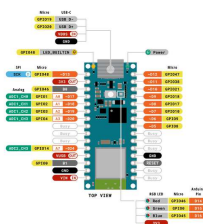
Arduino Nano ESP32, while the pair on the outside is the Power Connector, which provides optional power and mechanical support for custom expansions or circuits you might create.



Top face connectors

Nano Board Connector

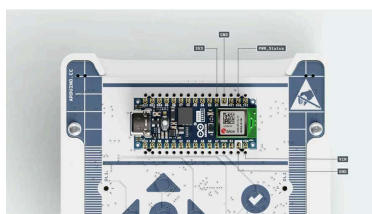
Because of the need to communicate with the STM board, some of the pins of the Arduino Nano ESP32 might be in use and, as such, are not recommended to be used by any other feature. This way, if you design a shield or attachment that interacts with these connectors, make sure to avoid the pins that are busy.



ESP Pinout

Power Connector

This connector allows you to get power from the Alvik's power system as well as get the power status. It also provides mechanical support for your designs. Here you can identify the connectors that can be used. Note that the remaining pins are not in use but can serve for mechanical support.



ON/OFF Switch



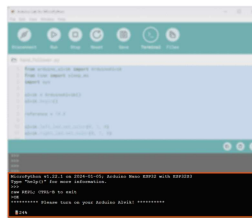
Switch on

At the back-right side of Alvik there is the main switch of the robot. When ON the robot will power up and it will execute the already loaded program.

Keep the robot off while programming to avoid undesired movements and remember to turn it on when you are ready to execute your program. A



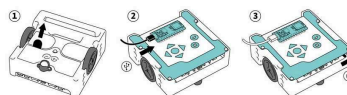
feature of the Arduino Lab for MicroPython terminal is that it will notify you if you forgot to switch on the robot to run the code.



Notification to switch on

Battery

Alvik's battery is protected by a pull-tab protective sticker. Remove the sticker to start working with the robot.



Alvik pull tab

Step 1: Locate the **pull-tab protective sticker** on the bottom of the robot. near the

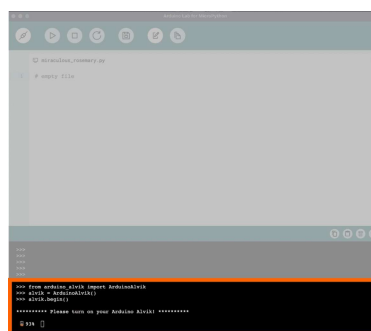
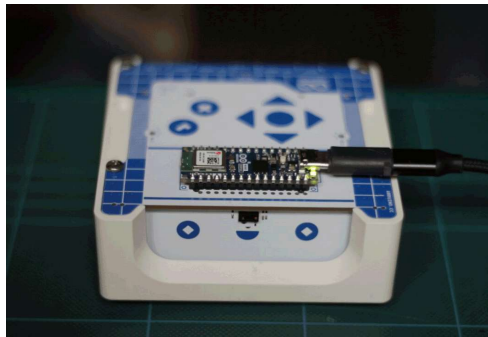
pull it upward firmly to remove the insulator and enable the battery connection.

Step 2: After the pull-tab is removed, turn the robot over, connect the robot to your computer and slide the power button to ensure it powers on.

The battery is a rechargeable Li-ion 18650, it allows to run Alvik for 8+ hours non stop.

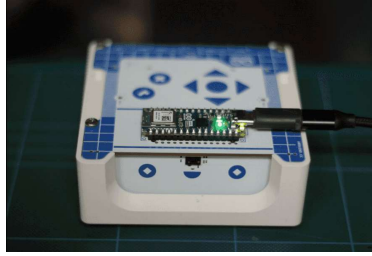
When you connect your Alvik to the computer, the Nano ESP32 reports the status of the battery through the terminal of the Arduino Lab for MicroPython and with its RGB status LED. In order to see the state of the battery, you need to call the `Alvik.begin()` function in any program or directly at the command line area.

When the battery is charging the status LED will blink RED for one second.



Terminal notification

When fully charged it will stay GREEN.



Alvik fully charged - Green LED



Don't confuse the RGB status LED with the power ON LED of the Nano ESP32, which is always green.

To preserve the battery life of your Alvik, always store it fully charged. **We recommend charging Alvik completely between working sessions.** If you won't be using it for an extended period, it's best to remove the battery until you're ready to use it again.

If Alvik is left unused for several months with the battery level below 20%, it may become fully depleted, leading to a voltage drop. In such cases, the battery will need to be charged externally using a compatible battery charger.

Replacing The Battery

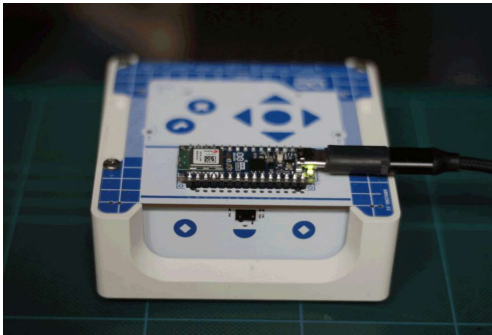
The battery is located in the bottom part of Alvik, if you need to access it you'll need to remove one Phillip's screw and take out the plastic holder.



Accessing battery compartment

Follow the steps below to replace Alvik's battery and reset the battery management system (BMS):

- 2 **Remove the existing battery:**
carefully take the old battery out of its compartment.
- 3 **Tighten the battery holder terminals** pull the + and – terminals slightly toward the center to ensure they make good contact with the battery terminals.
- 4 **Insert the new battery:** place the new battery into the holder, ensuring it is securely connected. To confirm proper contact, use a multimeter (set to 20V) to measure the battery holder terminals (+ & -); it should show the battery's voltage.
- 5 **Connect the Nano ESP32 to your computer** using the USB cable, while the Alvik switch is in OFF position.
- 6 **Switch ON Alvik**, like the image below.



i Note: When Alvik is switched back ON, the charging LED indicator goes OFF.

Alvik's Sensors

Alvik includes a set of different sensors listed below, all connected to the STM32 and accessible through the [APIs](#). For each sensor there is test example program that you can find in the *examples* folder in [this repository](#) for MicroPython and in [this repository](#) for C++.

Sensor name	Part name	Test program name
RGB		

Sensor name	Part name	Test program name
ToF 8x8 Array - up to 350 cm	LSM6DSOX	read_tof.py
IMU - 6 degree	VL53L7CX	read_imu.py
3x Line follower	custom made	line_follower.py
7x Touch sensor	AT42QT2120	read_touch.py

i

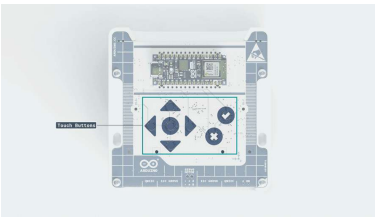
Before using the ToF sensor check if it has a yellow protective film, if present, remove it from the sensor to ensure it works properly

Touch Buttons

The Arduino Alvik robot is equipped with seven capacitive touch buttons. These buttons allow for interactive input and can be used to control various functions of the robot, such as movement, mode selection, and more.

Available buttons are:

- Up
- Down
- Left
- Right
- Center
- OK (represented by a "✓" icon)
- Cancel (represented by a "x")



Touch Button position

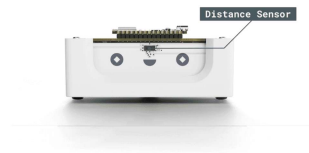
interface for user interaction.

Distance Sensor

The Arduino Alvik robot is equipped with a ToF (Time of Flight) 8x8 Array sensor, specifically the VL53L7CX, which can measure distances up to 350 cm. These distance sensors help the robot detect obstacles and measure the distance to objects in its environment. The sensor is integrated into the front PCB of the robot.

Key Features:

- Maximum distance: 350 cm
- High precision and accuracy
- Used for obstacle detection and distance measurement



Distance sensor position

Line Follower Sensors

The Arduino Alvik robot is equipped with line follower sensors that help it detect and follow lines on the ground. This is useful for applications where the robot needs to navigate along predefined paths. The line follower array consists of three phototransistors and five infrared LEDs, mounted on the front PCB.

The infrared LEDs emit light towards the ground, and the phototransistors detect the reflected light. When the sensors pass over a line (typically a dark line on a light surface), the amount of reflected light changes. Dark lines reflect less infrared light compared to light surfaces, allowing the phototransistors to detect the presence of a line

Detects lines on the ground (or any patterns using colors or materials with different reflective properties)

Helps in navigating predefined paths

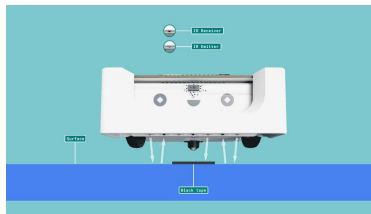
Consists of three phototransistors and five infrared LEDs

How it Works:

Infrared LEDs: Emit infrared light towards the ground. **Phototransistors:** Detect the reflected infrared light.

When over a dark line, the reflection is low, indicating the presence of a line.

When over a light surface, the reflection is high, indicating no line.



Follow the line sensor principles

By continuously reading the values from the phototransistors, the robot can determine its position relative to the line and adjust its movements to follow the path accurately.

Color Sensor

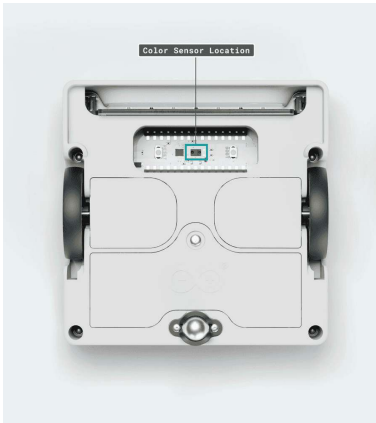
The color sensor on the Arduino Alvik robot is used to detect and identify colors on surfaces that the robot encounters. It provides both raw color readouts and labeled color information that can be used for various applications such as line following, object detection, and more. The sensor used is the APDS 9660.

Key Features:

Detects and identifies colors

Provides raw color readouts and labeled color information

Useful for line following, object detection, and other applications



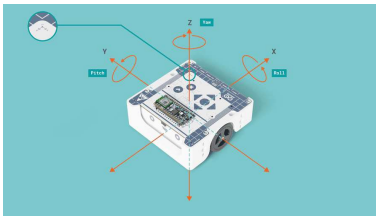
Color sensor bottom view

IMU

The Arduino Alvik robot is equipped with an onboard IMU (Inertial Measurement Unit) that provides valuable information about the robot's motion and orientation. The IMU can measure acceleration, angular velocity, and orientation. The sensor used is the LSM6DSOX.

Key Features:

- Measures acceleration, angular velocity, and orientation
- Provides roll, pitch, and yaw values
- Useful for motion tracking and stabilization



IMU axis reference

Alvik's Actuators

Actuator name	Part name	Test program name
Geared motors w/	GM12-N20VA-	wheels_positions.py

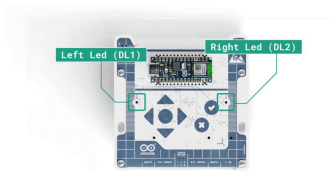
Actuator name	Part name	Test program name
	08255-150-EN	
RGB LEDs	RGB LEDs	leds_settings.py

Motors

The Alvik is equipped with two geared motors, specifically the GM12-N20VA-08255-150-EN. These motors allow for precise and reliable movement, essential for accurate navigation and control as each motor includes a magnetic relative encoder to ensure precise control and feedback of the motor's position.

- Gear Ratio:** 1:150
- Operating Voltage:** 6V
- No-load Speed:** 55 RPM
- No-load Current:** 70mA
- Encoder Type:** Magnetic relative encoder
- Magnetic relative AB encoder:** 3 ppr

LED Indicators



LED indicators position

The RGB LEDs can be used for visual feedback and can be programmed to display various colors and patterns. The LEDs take in **boolean** values for each color meaning that combinations are possible by setting each color **on** or **off** .

Out Of The Box Modes

just turn your Alvik ON, move the switch located at the bottom right corner of the robot to the right, wait until the LEDs turn blue and use the Up and Down buttons to pick one color, then hit the "tick" confirmation button.

Basic Touch Programming: Alvik's touch buttons can control actions such as movement, changing modes, and interacting with the environment. This allows for intuitive control schemes for different applications.

Following Objects: Using its distance sensors, Alvik can follow objects by maintaining a specific distance. This feature is useful for automated guided vehicles and interactive robotics.

Line Follower: Alvik's line follower sensors enable it to navigate along paths marked by lines on the ground, which is great for educational purposes and demonstrating basic robotic navigation.

For more information, visit the [Alvik Getting Started Guide](#).

Alvik's Firmware

To get started to play with Alvik you will need the following hardware and software:

Software Requirements

Operating Systems: Linux, macOS or Windows

[Arduino Lab for Micropython](#)

Updating Alvik's Brain (Nano ESP32)

It may happen that you used your Nano

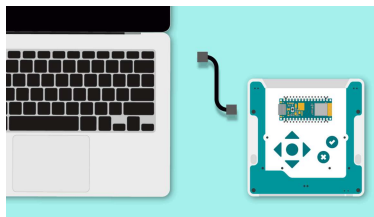
Nano ESP32 work with Alvik, there are few steps needed:

- 1 Install the MicroPython bootloader on it following [this guide](#).
- 2 Download the Alvik Micropython libraries Alvik MicroPython libraries from the [Alvik repository](#) ucPack libraries from the [ucPack repository](#)
- 1 Unzip both the downloaded libraries in a single "Alvik" folder, open the Arduino Lab for MicroPython, go to the "files" tab and set the path to the unzipped folder on the Arduino Lab for MicroPython.

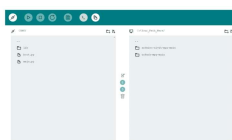


Setting the FW path on the Labs for micropython

- 2 Make sure your Alvik is OFF, connect it to your computer and then, turn it ON:

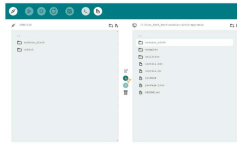


- 3 Connect your Alvik to the Arduino Labs for MicroPython and open the "lib".



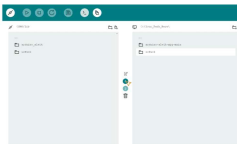
Setting the FW path on

- 4 Select the "Arduino-alkvik" and move it inside the "lib" folder in your Alvik.



Setting the FW path on
the Labs for
micropython

- 5 Go back to the main folder and select the "ucPack" folder found inside "ucPack-mpy-main" and move it next to the *arduino_alkvik* inside the "lib" folder in your Nano ESP32.



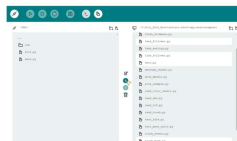
Setting the FW path on
the Labs for
micropython

- 6 Now go back to the main root of the files system on the Nano ESP32. Then in your local folder navigate to the examples folder once there, select the following files and move them to the main folder of the ESP32.

demo.py hand_follower.py

line_follower.py main.py

touch_move.py



Setting the FW path on
the Labs for
micropython

With this last step, your Nano ESP32 has been set up with the Alvik out of the box experience and is ready to be used

Updating Alvik's Body (STM32)

- 1 Download the [pre-compiled firmware](#) from the [Alvik Carrier GitHub repository](#)

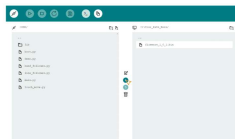
This step will download a "firmware_x_x_x.bin" file, save it in your Alvik folder.

- 2 Connect your Alvik to the Computer and to the Arduino Labs for MicroPython. Then, go to the files tab and navigate to the folder where you stored the "firmware_x_x_x.bin".



Setting the FW path on the Labs for micropython

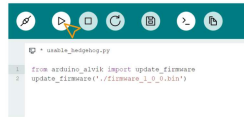
- 3 Let's move now the `firmware_x_x_x.bin` to the main root.



Setting the FW path on the Labs for micropython

- 4 Turn ON your alvik, go to the Editor tab and run the following commands by typing them and clicking on the "Play" button:

```
1 from arduino_alvik impo
2
3 update_firmware('./firmw
```



Setting the FW path on
the Labs for
micropython

After executing these commands, there will be updates of the process on the prompt, once the process finishes, the firmware of your alvik will be updated.

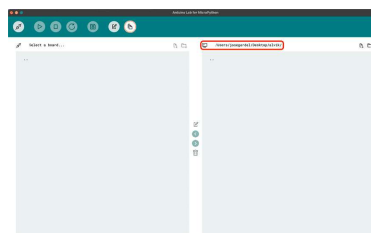
Hello Alvik! Your First Program!

Alvik is intended to be programmed with MicroPython. We recommend you to install the [Arduino Lab for MicroPython](#) editor.

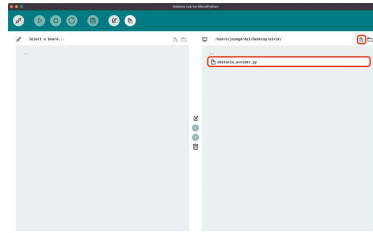
If you're using a Chromebook, you'll need to use the [online version] (<https://lab-micropython.arduino.cc/>) of Arduino Lab for MicroPython, using Chrome. You'll need to login with an Arduino account to use it. Note that some of the UI elements might look different check our [article](#) for more information.

Now that all the previous steps have been set, let's see how to create custom programs for Alvik to move forward until detecting an object in front of it, Alvik will detect it, dodge it and continue on its way.

1. Create an Alvik folder in your computer and set it as the path of the Arduino Lab for MicroPython IDE.

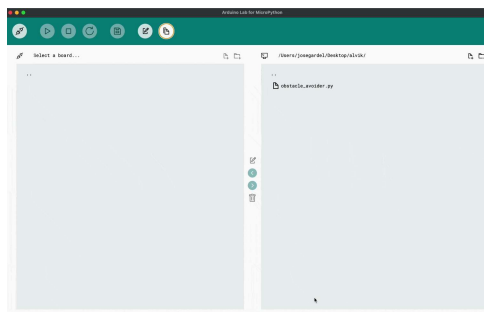


Adding Alvik folder path to the
IDE



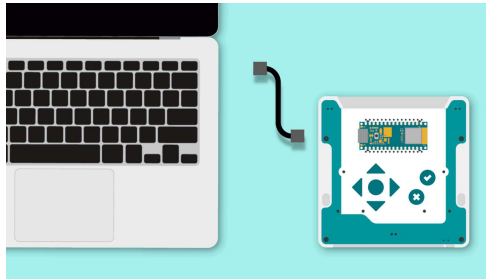
Creating obstacle_avoider.py file

3. Double click on the file to open it. Once it is opened, erase the text on it and add the following code.



```
1 from arduino_alvik import Ard
2 from time import sleep_ms
3 import sys
4
5 alvik = ArduinoAlvik()
6 alvik.begin()
7 sleep_ms(5000) #waiting for
8 #robot.set_illuminator(0)
9 distance = 10
10 speed = 40 #rpm
11
12 def turning():
13     alvik.set_wheels_speed(0,
14     sleep_ms(250)
15     alvik.set_wheels_speed(-3
16     sleep_ms(1500)
17     alvik.set_wheels_speed(35
18     sleep_ms(1000)
19
20 while (True):
21     distance_l, distance_cl,
22     sleep_ms(50)
23     print(distance_c)
24
25     if distance_c < distance:
26         turning()
27     elif distance_cl < distan
```

4. Connect Alvik to your PC using the cable included in the box, under the tray.



i Make sure that Alvik is OFF before connecting it to your computer.

5. Once Alvik is connected to the PC, connect it to the Arduino Lab for MicroPython and open the *main.py* file in the Alvik folder. Once the file is opened let's replace the `import demo` statement by `import obstacle_avoider`.

 A screenshot of the Arduino Lab for MicroPython IDE. The window title is "Arduino Lab for MicroPython". The file explorer on the left shows a folder named "obstacle_avoider.py". The main editor area displays the contents of this file, which includes imports for "obstacle_avoider", "time", and "sys", followed by a series of comments and code for controlling the robot's movement based on sensor data.

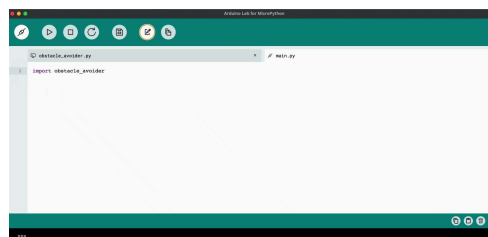

```

1 from obstacle_avoider import obstacle_avoider
2 from time import sleep_ms
3 import sys
4
5 alvik = Alvik()
6 alvik.begin()
7 sleep_ms(1000) #waiting for the robot to setup
8 motor_pin_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100]
9 distance = 0
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
  
```

If you want to go back to the out of the box experience where you could select between reg, green and blue programs,

i you only need to modify the *main.py* again replacing the `import obstacle_avoider` statement by `import demo`

6. The last step is to move the *obstacle_avoider.py* file from the local repository to Alvik's memory.



You are now all set, disconnect Alvik from the computer, put some obstacles around Alvik, turn it ON and see how Alvik detects them and turns to avoid them.

Deep Dive Programming Alvik

Controlling the Motors

Movement is one of Alvik's main features, making it essential to have a variety of methods to control Alvik's motors. This flexibility allows you to use different control methods depending on your needs, whether it's for precise movement or simple speed control.

Power Over Distance/Time

You can set the speed of Alvik's motors directly and let it run for a specific amount of time or distance. This method is straightforward and useful for simple tasks where precise control isn't necessary.

Function: `set_wheels_speed`

The `set_wheels_speed` function sets the speed of the left and right motors.

Inputs:

`left_speed`: The speed value for the left motor.

`right_speed`: The speed value for the right motor.

`unit`: Unit of rotational speed of the wheels (default is 'rpm').

Example Usage:

```
1 alvik.set_wheels_speed(30, 30,  
2 sleep_ms(2000) # Run motors at  
3 alvik.brake()
```

Alvik allows you to control the motors based on specific distances or angles. This method provides precise control over the robot's movements, ensuring it travels the exact distance or rotates to the exact angle specified.

`set_wheels_position` method

The `set_wheels_position` function sets the angle position for the left and right motors.

Inputs:

`left_angle`: The angle value for the left motor.

`right_angle`: The angle value for the right motor.

`unit`: The angle unit (default is 'deg').

Example Usage:

```
1 alvik.set_wheels_position(360, 360, 'deg')
2 sleep_ms(2000) # Rotate wheels
3 alvik.brake()
```

Using Move

The `move` function moves the robot by a given distance.

Inputs:

`distance`: The distance value.

`unit`: The distance unit (default is 'cm').

`blocking`: True or False, whether the function should block until the movement is complete (default is True).

Example Usage:

```
1 alvik.move(50, 'cm')
2 sleep_ms(2000) # Move forward
3 alvik.brake()
```

Using Drive

You can also specify the power and time for Alvik's motors in terms of degrees per second or centimeters per second. This method is useful for tasks requiring fine-tuned control of the robot's speed and direction.

`drive` method

The `drive` function drives the robot by setting the linear and angular velocity.

Inputs: `linear_velocity`: Speed of the robot. `angular_velocity`: Speed of the wheels. `linear_unit`: Unit of linear velocity (default is 'cm/s'). `angular_unit`: Unit of rotational speed of the wheels (default is 'deg/s').

Example Usage:

```
1 alvik.drive(10, 30, 'cm/s', 'deg/s')
2 sleep_ms(2000) # Drive at 10 cm/s for 2 seconds
3 alvik.brake()
```

Encoder's Control

The Alvik robot provides various functions to control and monitor the motors using encoders. These functions allow you to get real-time feedback on the speed, position, and pose of the robot, which is essential for precise movement and navigation.

Check Wheels Speed

The `get_wheels_speed` function returns the current speed of the wheels.

Inputs:

`unit`: Unit of rotational speed of the wheels (default is 'rpm').

Outputs:

`left_wheel_speed`: The speed of the

`right_wheel_speed`: The speed of the right wheel.

Example Usage:

```
1 left_speed, right_speed = alvik
2 print(f"Left Wheel Speed: {left_speed}, Right Wheel Speed: {right_speed}")
```

Check Wheels Position

The `get_wheels_position` function returns the current angle of the wheels.

Inputs:

`unit`: Unit of the wheel angle (default is 'deg').

Outputs:

`left_wheel_angle`: The angle of the left wheel.

`right_wheel_angle`: The angle of the right wheel.

Example Usage:

```
1 left_angle, right_angle = alvik.get_wheels_position()
2 print(f"Left Wheel Angle: {left_angle}, Right Wheel Angle: {right_angle}")
```

Check Velocity

The `get_drive_speed` function returns the linear and angular velocity of the robot.

Inputs:

`linear_unit`: Unit of linear velocity (default is 'cm/s').

`angular_unit`: Unit of rotational speed of the wheels (default is 'deg/s').

Outputs:

`linear_velocity`: The linear speed of the robot.

`angular_velocity`: The angular speed of the wheels.

```
1 linear_velocity, angular_velocity = alvik.get_velocity()
2 print(f"Linear Velocity: {linear_velocity} cm/s, Angular Velocity: {angular_velocity} deg/s")
```

Check Pose

The `get_pose` function returns the current pose of the robot.

Inputs:

`distance_unit`: Unit of x and y outputs (default is 'cm').

`angle_unit`: Unit of theta output (default is 'deg').

Outputs:

`x`: The x-coordinate of the robot.

`y`: The y-coordinate of the robot.

`theta`: The orientation angle of the robot.

Example Usage:

```
1 x, y, theta = alvik.get_pose(distance_unit='cm', angle_unit='deg')
2 print(f"Pose - X: {x} cm, Y: {y} cm, Theta: {theta} deg")
```

These functions allow for precise control and monitoring of the Alvik robot's movements, enabling advanced navigation and path planning capabilities.

Reading Buttons

The Arduino Alvik robot is equipped with several touch buttons that can be used for various input purposes. The following functions are available to read the state of each button:

Detect **any** button pressed.

`get_touch_any` function returns true if any of the buttons is pressed.

`get_touch_ok` function returns true if the OK button is pressed.

Detect the **Cancel** button pressed.

`get_touch_cancel` function returns true if the Cancel button is pressed.

Detect the **Center** button pressed.

`get_touch_center` function returns true if the Center button is pressed.

Detect the **Up** button pressed.

`get_touch_up` function returns true if the Up button is pressed.

Detect the **Left** button pressed.

`get_touch_left` function returns true if the Left button is pressed.

Detect the **Down** button pressed.

`get_touch_down` function returns true if the Down button is pressed.

Detect the **Right** button pressed.

`get_touch_right` function returns true if the Right button is pressed.

Example The following example demonstrates how to read the state of each button and print its name when pressed:




```

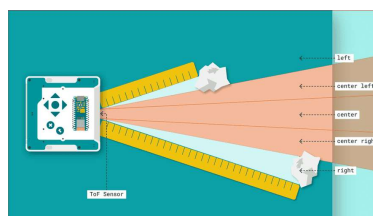
1 from arduino_alvik import Ard
2 from time import sleep_ms
3 import sys
4
5 alvik = ArduinoAlvik()
6 alvik.begin()
7
8 while True:
9     try:
10         if alvik.get_touch_an
11             if alvik.get_touc
12                 print("UP")
13             if alvik.get_touc
14                 print("DOWN")
15             if alvik.get_touc
16                 print("LEFT")
17             if alvik.get_touc
18                 print("RIGHT")
19             if alvik.get_touc
20                 print("OK")
21             if alvik.get_touc
22                 print("CANCEL
23             if alvik.get_touc
24                 print("CENTER
25
26         sleep_ms(100)
27     except KeyboardInterrupt:
28         print('over')

```

This example provides a simple way to read and respond to button presses on the Arduino Alvik robot. Feel free to expand to it by adding customized responses to each different press.

Detecting Obstacles

These sensors provide detailed readouts from multiple positions around the robot, including **left**, **center-left**, **center**, **center-right**, **right**, **top**, and **bottom**. Unlike single-point sensors, these distance sensors offer comprehensive zone-based information.



ToF Sensor top-view

The distance sensor array is divided into five distinct zones: left, center-left, center, center-right, and right. Each zone measures distances and is capable of detecting objects up to three meters away. It's important to note that while the sensors can detect objects at greater distances, the reliability of the readings may decrease.

Read distance

The `get_distance` function returns the distance readouts from the left, center-left, center, center-right, and right sensors.

Outputs: `L`: Left sensor readout. `CL`: Center-left sensor readout. `C`: Center sensor readout. `CR`: Center-right sensor readout. `R`: Right sensor readout.

Example Usage

Here is an example of how to use the distance sensors to get distance measurements:

```
1 from arduino_alvik import ArduinoAlvik
2 from time import sleep_ms
3 import sys
4
5 alvik = ArduinoAlvik()
6 alvik.begin()
7
8 while True:
9     try:
10         L, CL, C, CR, R = alvik.get_distance()
11         T = alvik.get_distance()
12         B = alvik.get_distance()
13         print(f'T: {T} | B: {B}')
14         sleep_ms(100)
15     except KeyboardInterrupt as e:
16         print('over')
17         alvik.stop()
18         sys.exit()
```

In this example, the robot uses its distance sensors to measure distances in its environment. The distance readouts from the left, center-left, center, center-right, right, top, and bottom sensors are printed

Following a Line

Read line sensor

The `get_line_sensors` function returns the line follower sensors' readouts, providing the status of the left, center, and right sensors.

Outputs:

`left` : Left sensor readout.

`center` : Center sensor readout.

`right` : Right sensor readout.

Example Usage

Here is an example of how to use the line follower sensors to make the robot follow a line:

```
1 from arduino_alvik import Ard
2 from time import sleep_ms
3 import sys
4
5 def calculate_center(left: in
6     centroid = 0
7     sum_weight = left + cente
8     sum_values = left + 2 * c
9     if sum_weight != 0:
10         centroid = sum_values
11         centroid = 2 - centro
12     return centroid
13
14 alvik = ArduinoAlvik()
15 alvik.begin()
16
17 error = 0
18 control = 0
19 kp = 50.0
20
21 alvik.left_led.set_color(0, 0
22 alvik.right_led.set_color(0,
23
24 while alvik.get_touch_ok():
25     sleep_ms(50)
26
27 while not alvik.get_touch_ok(
28     sleep_ms(50)
```

In this example, the robot uses its line

determines the center of the line's position, and the robot adjusts its wheel speeds to stay on the line based on the calculated error from this center.

Sensing Colors

Available features

- 1 **Calibrate** the sensor for a specific background

The `color_calibration` function is used to calibrate the color sensor for accurate color detection. Calibration can be done against a white or black background.

```
1 color_calibration(backg
```

Inputs: `background`: A string specifying the background color for calibration. Can be "white" or "black".

- 2 Gets the **raw** sensor value

The `get_color_raw` function returns the raw readout from the color sensor. This is the direct sensor data before any processing or labeling. It can be useful for advanced applications where precise color data is needed.

```
1 get_color_raw()
```

Outputs: `color`: The color sensor's raw readout.

- 3 Gets the **labelled** sensor value

The `get_color_label` function

function processes the raw sensor data and converts it into a human-readable label, such as "red", "blue", "green", etc. It simplifies the use of color data for most applications however some flexibility is lost as colors are grouped into the nearest labeled color.

```
1 get_color_label()
```

Outputs: `color`: The label of the color as recognized by the sensor. These can be: `'BLACK'`, `'GREY'`, `'BLACK'`, `'LIGHT GREY'`, `'WHITE'`, `'YELLOW'`, `'LIGHT GREEN'`, `'BLUE'`, `'VIOLET'`, `'BROWN'`, `'ORANGE'`, `'RED'`.

Example Usage

Here is an example of how to use the `get_color_label` function in a script that makes the robot walk in a straight line, detect three different colors (ignoring white), stop, and communicate the detected colors via serial every second:

```

1 from arduino_alvik import
2 import time
3 from time import sleep
4
5 # Initialization
6 alvik = ArduinoAlvik(
7 alvik.begin()
8 sleep(5) # Waiting f
9
10 # Calibrate color sen
11 alvik.color_calibrati
12
13 # Main logic
14 detected_colors = set
15
16 print("Starting to mo
17
18 try:
19     while len(detected
20         alvik.set_whe
21         color = alvik
22
23         if color != '
24             detected_
25             print(f"D
26
27         time.sleep(0.
28

```

4 get_color

The `get_color` function returns the normalized color readout of the color sensor. This function can output the color in either RGB or HSV format.

```

1 get_color(color_format:

```

Inputs: `color_format`: The format of the color readout. Can be "rgb" or "hsv".

Outputs: `r` or `h`: The red component in RGB format or the hue in HSV format. `g` or `s`: The green component in RGB format or the saturation in HSV format. `b` or `v`: The blue component in

You can use these functions depending on your needs:

Use `color_calibration` to calibrate the sensor for accurate readings.

Use `get_color_raw` to get the raw sensor data.

Use `get_color_label` to get a human-readable label of the detected color.

Use `get_color` to get normalized color data in RGB or HSV format.

Use `hsv2label` to convert HSV values to a color label.

Detecting Falling and Crashes (IMU)

The `get_imu` function from the [Alvik API](#) returns all the IMU's readouts. The readouts include acceleration (ax, ay, az) and angular acceleration (gx, gy, gz) on the x, y, and z axes.

Available Functions

- 1 Retrieves IMU's orientation values like **roll**, **pitch** and **yaw**

The `get_orientation()` function returns the orientation of the IMU, including roll, pitch, and yaw values. This can be useful for determining the robot's orientation in 3D space.

Outputs: **r:** roll value **p:** pitch value **y:** yaw value

- 2 Retrieves 3-axial acceleration values

The `get_accelerations()` function returns the 3-axial acceleration of the IMU. This provides the acceleration values along the x, y, and z axes.

Outputs: **ax:** acceleration on x **ay:** acceleration on y **az:** acceleration

- 3 Retrieves 3-axial angular acceleration values

The `get_gyros()` function returns the 3-axial angular acceleration of the IMU. This provides the angular acceleration values along the x, y, and z axes.

Outputs: **gx:** angular acceleration on x **gy:** angular acceleration on y **gz:** angular acceleration on z

- 4 Retrieves all IMU readouts including acceleration and angular acceleration

The `get_imu()` function returns all the IMU's readouts, including both the acceleration and angular acceleration values.

Outputs: **ax:** acceleration on x **ay:** acceleration on y **az:** acceleration on z **gx:** angular acceleration on x **gy:** angular acceleration on y **gz:** angular acceleration on z

Example

Here is a combined example that utilizes all IMU functions:




```
1 from arduino_alvik import Ard
2 import time
3
4 # Initialization
5 alvik = ArduinoAlvik()
6 alvik.begin()
7
8 # Give some time for initiali
9 time.sleep(2)
10
11 # Get orientation
12 orientation = alvik.get_orien
13 roll = orientation['r']
14 pitch = orientation['p']
15 yaw = orientation['y']
16 print(f"Orientation - Roll: {
17
18 # Get accelerations
19 accelerations = alvik.get_acc
20 ax = accelerations['ax']
21 ay = accelerations['ay']
22 az = accelerations['az']
23 print(f"Acceleration - X: {ax
24
25 # Get gyros
26 gyros = alvik.get_gyros()
27 gx = gyros['gx']
28 gy = gyros['gy']
```

Now that we've got a solid grasp on how the IMU works and how to interact with it, let's put this knowledge to use in a practical scenario. Events like falls and crashes are critical for a robot, as they often result from sudden changes in linear acceleration (movement along the x, y, or z axes) or angular velocity (rotational movement around these axes). By monitoring these specific changes, we can effectively detect such incidents and respond accordingly.

In the following example, we'll create a sketch that uses the IMU to identify falls and crashes. The sketch will look for sudden spikes in linear acceleration or angular velocity, using set thresholds to accurately determine when these events occur.

Here's how you can implement this functionality:

```
1 from arduino_alvik import Ard
2 import time
3
4 # Initialize the Alvik robot
5 alvik = ArduinoAlvik()
6 alvik.begin()
7
8 # Function to detect falling
9 def detect_fall_or_crash():
10     imu_readings = alvik.get_
11
12     ax = imu_readings['ax']
13     ay = imu_readings['ay']
14     az = imu_readings['az']
15     gx = imu_readings['gx']
16     gy = imu_readings['gy']
17     gz = imu_readings['gz']
18
19     # Thresholds for detectin
20     acceleration_threshold =
21     angular_acceleration_thre
22
23     if abs(ax) > acceleration
24         print("Fall or crash
25
26     if abs(gx) > angular_acce
27         print("Fall or crash
28
```

Now that you've got the basics down, why not take it a step further? You can customize how Alvik reacts to falls and crashes. Whether it's sending an alert, logging the event for later analysis, or performing a recovery maneuver, the possibilities are endless.

And let's be honest, there's a certain satisfaction in programming your robot to handle mishaps gracefully (or even dramatically). It's not just about functionality—sometimes, it's about giving our robots that extra bit of personality and, let's face it, feeling a bit superior when our creations stumble. Go ahead, have fun with it and see what creative solutions you can come up with when Alvik takes a tumble!

LEDs

The Arduino Alvik robot comes equipped with two RGB LEDs that can be controlled to

used for signaling, indicating status, or just for fun. The following functions are available for controlling the LEDs:

Set color of the Alvik's RGB LEDs

The `set_color` function sets the color of the LED by specifying the intensity of red, green, and blue components.

Inputs: `red`: Intensity of the red component (0 or 1) `green`: Intensity of the green component (0 or 1) `blue`: Intensity of the blue component (0 or 1)

In this case the LEDs take in **boolean** values for each color meaning that combinations are possible by setting each color **on** or **off** instead of controlling intensity.

Example

Here's an example sketch that cycles through different colors on both LEDs:

```
1 from arduino_alvik import ArduinoAlvik
2 from time import sleep
3 import sys
4
5 alvik = ArduinoAlvik()
6 alvik.begin()
7 sleep(5)
8
9 while True:
10     alvik.left_led.set_color(0, 0, 0)
11     alvik.right_led.set_color(0, 0, 0)
12     sleep(1)
13     alvik.left_led.set_color(0, 0, 1)
14     alvik.right_led.set_color(0, 0, 1)
15     sleep(1)
16     alvik.left_led.set_color(0, 1, 0)
17     alvik.right_led.set_color(0, 1, 0)
18     sleep(1)
19     alvik.left_led.set_color(1, 0, 0)
20     alvik.right_led.set_color(1, 0, 0)
21     sleep(1)
```

By following these steps, you can control the LEDs on the Arduino Alvik robot to display different colors and create various lighting effects.

Talking With Other Devices!

WiFi

The ESP32 on the Arduino Alvik robot includes built-in Wi-Fi capabilities, enabling it to connect to wireless networks and communicate with other devices over the internet. This can be particularly useful for remote control applications. In this example, we'll set up the Alvik robot to connect to a Wi-Fi network, host a web server, and provide a simple web interface with buttons to control the robot's movements.

The provided code will:

- 1 Connect the ESP32 to a specified Wi-Fi network.
- 2 Set up a web server to listen for incoming HTTP requests.
- 3 Serve a simple web page with buttons for directional control.
- 4 Move the robot in the specified direction when a button is pressed.

Step By Step Setup

- 1 Replace `Insert_SSID_Here` and `Password_Here` with your Wi-Fi credentials.
- 2 Upload the script to the Arduino Nano ESP32.
- 3 Connect to the robot's IP address (printed on the console) using a web browser.
- 4 Use the web interface to control the robot's movement.

Connecting to Wi-Fi

The following code connects the ESP32 to a specified Wi-Fi network using the provided SSID and password.

```
1 import network
2 import socket
3 from arduino_alvik import ArduinoAlvik
4 from time import sleep_ms
5
6 # Wi-Fi credentials
7 SSID = "Insert_SSID_Here"
8 PASSWORD = "Password_Here"
9
10 # Connect to Wi-Fi
11 sta_if = network.WLAN(network.STA_IF)
12 sta_if.active(True)
13 sta_if.connect(SSID, PASSWORD)
14
15 # Wait for connection
16 while not sta_if.isconnected():
17     pass
18
19 print("Connected to WiFi. IP address is", sta_if.ifconfig()[0])
```

Replace `Insert_SSID_Here` and `Password_Here` with your Wi-Fi network's SSID and password to enable the ESP32 to connect to your Wi-Fi.

Initializing the Robot

The following code initializes the Alvik robot and gives it time to set up.

```
1 # Initialize the robot
2 alvik = ArduinoAlvik()
3 alvik.begin()
4 sleep_ms(5000)
```

Setting Up the Web Server

The following code defines the HTML for the web interface. Feel free to customize this to your taste.

```

1 # HTML for the web interface
2 html = ""
3 <!DOCTYPE html>
4 <html>
5 <head>
6 <title>Alvik Robot Control</title>
7 </head>
8 <body>
9 <h1>Control Alvik Robot</h1>
10 <form action="/up">
11     <button type="submit">Up</button>
12 </form>
13 <form action="/down">
14     <button type="submit">Down</button>
15 </form>
16 <form action="/left">
17     <button type="submit">Left</button>
18 </form>
19 <form action="/right">
20     <button type="submit">Right</button>
21 </form>
22 </body>
23 </html>
24 ""

```

We finally set up the web server, note that the selected port should be valid for your network with no conflicts. This can be tuned to the setting that best works for your network configuration.

```

1 # Specify a port
2 port = 8080
3 addr = socket.getaddrinfo('0.0.0.0', port)
4 s = socket.socket()
5 s.bind(addr)
6 s.listen(5)
7
8 s.settimeout(5)
9
10 print(f'Listening on {sta_if.ip}')

```

Handling HTTP Requests

The following code handles incoming HTTP requests and moves the robot in the specified direction. Note that the robot will continue the action until the break is pressed.

```

1 def handle_request(conn):
2     try:
3         request = conn.recv(1
4         first_line = request.
5         path = first_line.spl
6
7         # Strip query strings
8         if '?' in path:
9             path = path.split
10        print(f"Request path:
11
12        # Ignore favicon requ
13        if path == '/favicon.
14            conn.send('HTTP/1
15            conn.send('Conne
16            return
17
18        # Control the robot b
19        if path == '/up':
20            alvik.set_wheels_
21        elif path == '/down':
22            alvik.set_wheels_
23        elif path == '/left':
24            alvik.set_wheels_
25        elif path == '/right'
26            alvik.set_wheels_
27        else:
28            alvik.brake()

```

Main Loop

The following code runs the main loop to handle incoming connections and process HTTP requests.

```

1 # Main loop to handle incoming
2 try:
3     while True:
4         try:
5             conn, addr = s.accept
6             print('Connection
7             handle_request(conn
8         except OSError as e:
9             if e.errno == 116:
10                print("Waiting
11            elif e.errno == 104
12                print("Connect
13            else:
14                print(f"Accept
15 except KeyboardInterrupt:
16     print("Server stopped")
17     s.close()
18     alvik.stop()

```

This section ensures the server keeps running, accepting incoming connections, and handling requests until the script is interrupted. If an `EADDRINUSE` error occurs, change the port number from 80 to an unused port, like 8080, in the

```
addr = socket.getaddrinfo('0.0.0.0',
80)[0][-1]
```

line. Also, make sure the device you access the web interface on should be on the same network as the Alvik.

Full Code

```
1 import network
2 import socket
3 from arduino_alvik import Ar
4 from time import sleep_ms
5
6 # Wi-Fi credentials
7 SSID = "Insert_SSID_Here"
8 PASSWORD = "Insert_Password_"
9
10 # Connect to Wi-Fi
11 sta_if = network.WLAN(network
12 sta_if.active(True)
13 sta_if.connect(SSID, PASSWOR
14
15 # Wait for connection
16 while not sta_if.isconnected
17     pass
18
19 print("Connected to WiFi. IP
20
21 # Initialize the robot
22 alvik = ArduinoAlvik()
23 alvik.begin()
24 sleep_ms(5000)
25
26 # HTML for the web interface
27 html = ""
28 <!DOCTYPE html>
```

ESP-NOW

The ESP32 on the Arduino Alvik robot also supports ESP-NOW, a fast, connectionless communication protocol that enables direct, quick, and low-power control of smart devices without the need for a router. ESP-

applications. It supports the ESP8266, ESP32, ESP32-S, and ESP32-C series. In this example, we'll set up the Alvik robot to receive commands via ESP-NOW.

Device Identification

It is important for ESP-NOW to ensure that you know the MAC address for your device.

You can do that by running the following code:

```
1 import network
2
3 # Initialize the network interface
4 e = network.WLAN(network.STA_IF)
5 e.active(True)
6
7 # Get and print the MAC address
8 mac = e.config('mac')
9 print("MAC address:", ':'.join(
```

Each device will have a different address, feel free to identify your devices so that you can easily build your ESP-NOW projects.

Now that you know the address for your device we can run your first ESP-NOW project.

Lets now establish connection between a pair of Alviks. One will act as a controller and the other will follow the controls and move in the according direction.

The provided code will:

- 1 Initialize ESP-NOW on the ESP32.
- 2 Set up a callback function to handle incoming ESP-NOW messages.
- 3 Move the robot based on the received commands while buttons are pressed.

Step By Step Setup

- 2 Use another ESP32 device to send commands to the Alvik robot via ESP-NOW.

Setting Up ESP-NOW on the Receiver

The following code initializes ESP-NOW on the receiver and sets up a callback function to handle incoming messages.

```
1 from arduino_alvik import Ard
2 import network
3 import espnow
4
5 # Initialize ESP-NOW
6 wlan = network.WLAN(network.S
7 wlan.active(True)
8 esp = espnow.ESPNow()
9 esp.active(True)
10
11 # Print the MAC address
12 mac = wlan.config('mac')
13 print("MAC address:", ':'.joi
14
15 # Initialize Alvik
16 alvik = ArduinoAlvik()
17 alvik.begin()
18
19 def perform_action(command):
20     print(f"Performing action
21     if command == "MOVE_FORWA
22         alvik.set_wheels_spee
23     elif command == "MOVE_BAC
24         alvik.set_wheels_spee
25     elif command == "TURN_LEF
26         alvik.set_wheels_spee
27     elif command == "TURN_RIG
28         alvik.set_wheels_spee
```

Setting Up ESP-NOW on the Transmitter

The following code can be used on another Alvik to send commands to the Alvik robot.

```

1  from arduino_alvik import Ard
2  from time import sleep_ms
3  import network
4  import espnow
5
6  # Initialize ESP-NOW
7  wlan = network.WLAN(network.S
8  wlan.active(True)
9  esp = espnow.ESPNow()
10 esp.active(True)
11
12 # Define the peer's MAC addre
13 peer_mac = b'\x74\x4d\xbd\x00
14
15 # Try to add the peer, and ha
16 try:
17     esp.add_peer(peer_mac)
18 except OSError as e:
19     if e.args[0] == -12395:
20         print("Peer already e
21     else:
22         raise
23
24 # Initialize Alvik
25 alvik = ArduinoAlvik()
26 alvik.begin()
27
28 # Define button to command ma

```

Receiver behaviour

- 1 **Initializing ESP-NOW:** The code sets up the ESP32 as a Wi-Fi station and initializes ESP-NOW.
- 2 **Callback Function:** A callback function `on_receive_msg` is defined to handle incoming messages. This function decodes the message and moves the robot based on the received command.
- 3 **Registering the Callback:** The callback function is registered with ESP-NOW to ensure it gets called when a message is received.

Transmitter behaviour

- 1 **Initializing ESP-NOW:** The code sets up the ESP32 as a Wi-Fi station and initializes ESP-NOW.
- 2 **Adding a Peer:** The MAC address of the receiver (Alvik robot) is added as a peer.
- 3 **Sending Commands:** The function

commands are provided to move the robot in different directions.

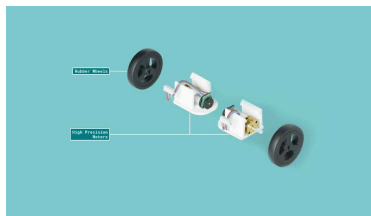
By following these steps, you'll be able to control your Arduino Alvik robot remotely using ESP-NOW, a fast and efficient communication protocol. Feel free to tinker with this feature to add new and interesting interactions.

Maintenance

Check Wheels Alignment

The motion calculations performed by Alvik are based on the precise position of both wheels. You can check this position and correct it in case you think your robot is not moving as expected:

- 1 Place your Alvik on a side on a flat surface
- 2 Check that the exterior part of the wheel is in contact with the flat surface and aligned with the plastic chassis
- 3 Repeat with the other wheel



Check wheel alignment

Calibrate Color Sensors

To calibrate the color sensor, placed in the bottom PCB under the Nano ESP32, you'll need a white surface and a black surface. Follow these steps:

- 1 Open the Arduino Lab for MicroPython
- 2 Connect Alvik

Now you're ready to send commands directly to Alvik. Every time you click enter the command will be executed in real time.

```
1 from arduino_alvik import ArduinoAlvik
2
3 alvik = ArduinoAlvik()
4
5 alvik.begin()
```

Now place your robot on the white surface and type:

```
1 alvik.color_calibration('white')
```

Now place your robot on the black surface and type:

```
1 alvik.color_calibration('black')
```

Press reset on Lab for MicroPython.

You can now test using `read_color_sensor.py` in the examples folder. Refer to the [test chapter](#) if you have any problems.



Colors are tested on paper painted using acrylic marker pens, such as UNIPOSKA, or paper printed with an inkjet printer.

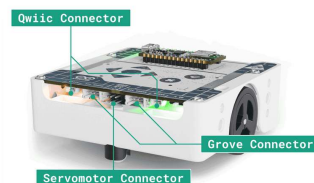
Expanding the Robot

Additional Connectors

Alvik includes several connectors that expand its capabilities:

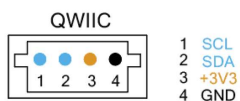
Servomotor Connectors

Lego Technic Compatibility



Connector positions

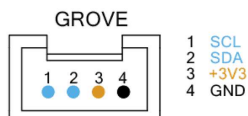
Qwiic Connectors



QWIIC Connector

The Qwiic connectors on the Arduino Alvik robot are used for I2C connectivity with compatible sensors and actuators using the Qwiic standard. This standard simplifies the process of connecting multiple I2C devices by using a common connector and pinout. The Qwiic system supports daisy-chaining, allowing multiple devices to be connected in series. There are two Qwiic connectors, one on each side at the back of the robot, allowing for easy expansion and integration of additional components.

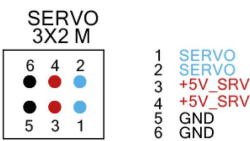
Grove Connectors



Grove Connector

The Grove connectors on the Arduino Alvik robot are used for connectivity with compatible sensors and actuators following the Grove standard. The Grove system uses standardized connectors and pinouts to simplify the connection of various I2C devices. The system is modular and user-friendly, ideal for rapid prototyping and educational purposes. There are two Grove connectors, one on each side at the back of the robot, providing additional flexibility for expanding the robot's capabilities.

Servomotor Connectors



Servo Connector

The Arduino Alvik robot features 3-pin connectors for attaching servomotors while providing PWM control external servomotors. The connectors are disposed on a 6 by 2 header in the center at the back of the robot, supporting multiple servomotor connections. This arrangement enables complex robotic movements and functions, allowing the robot to perform a variety of additional tasks.

Lego Technic Compatibility

Alvik's body includes compatibility with Lego Technic pieces for physical expansion and customization.

Custom Parts

 When adding extensions to the robot, never use screws longer than 10 mm or the device could be damaged.



Threads for custom extensions

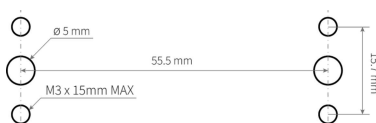
Add LEGO® Addons

On both sides of Alvik there are different housings that let you add:

4x M3 screws per side

2x LEGO® Technic™ Connector per side

The dimensions are:

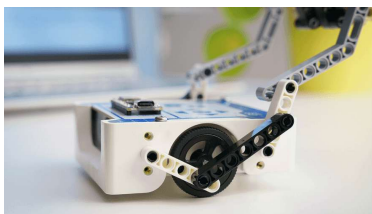


Addons dimensions

As reference you can take a look at the following images:



Alvik screws compatibility
connectors



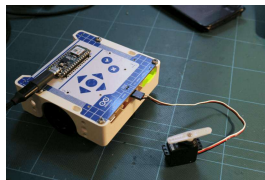
Alvik LEGO® compatibility
connectors

The servo motors connectors are placed at the back of Alvik, in this tutorial we'll attach a servo motor to the port A. You can take a look at the pinout image in the [pinout](#) chapter.



The port provides 5 Volt to the motor, so be sure to connect a servo that runs with 5V.

- 1 Connect the servo motor to the upper port



Servo connected

- 2 Open the Arduino Lab for MicroPython
- 3 Connect Alvik with a USB cable and click CONNECT
- 4 Copy and paste the following test code

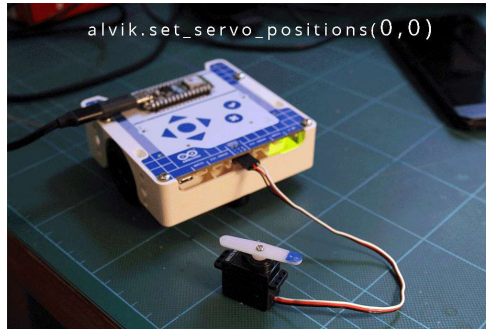
```
1 from arduino_alvik import Ardu
2 import time
3
4 # Initialize Alvik
5 alvik = ArduinoAlvik()
6 alvik.begin()
7
8 while True:
9     # Moves servo
10    alvik.set_servo_positions(0
11    time.sleep(2)
12    alvik.set_servo_positions(9
13    time.sleep(2)
14    alvik.set_servo_positions(1
15    time.sleep(2)
16    alvik.set_servo_positions(9
17    time.sleep(2)
```

- 1 Click on the PLAY button to run

- 2 The motor should move as in the gif below

Click on the PLAY button to run the test code

The motor should move as in the gif below



If you want to understand how the command `alvik.set_servo_positions` works, you can have a look in the [API overview](#).

Add I2C Grove

The I2C Grove connectors are placed at the back of Alvik, in this tutorial we'll see how to scan a generic I2C device connected to it. You can take a look at the pinout image at [Alvik's Product Page](#).

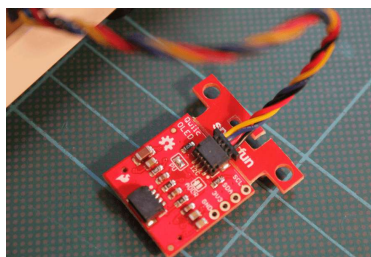
- 1 Connect the I2C Grove device to one of the two ports.
- 2 Open the Arduino Lab for MicroPython
- 3 Connect Alvik with a USB cable and click CONNECT
- 4 Turn ON Alvik
- 5 Copy and paste the following test code

```
1 from machine import I2C
2 from machine import Pin
3
4 i2c = I2C(0, scl=Pin(12), Pin.0)
5
6 print()
7 print('Scan i2c bus...')
8 print()
9
10 devices = i2c.scan()
11
12 if len(devices) == 0:
13     print("No i2c device !")
14 else:
15     print('i2c devices found:')
16 print()
17
18 for device in devices:
19     print("Decimal address: ", device)
20
21 print()
```

- 1 Click on the PLAY button to run the test code
- 2 Look at the terminal to see the list of the I2C devices

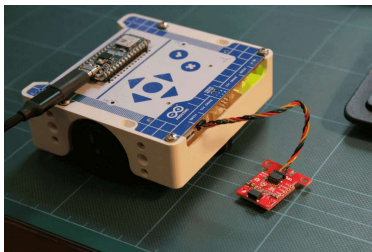
Add Qwiic

The Qwiic connectors are placed at the back of Alvik, for this example we'll be using the Qwiic OLED display from SparkFun. You can take a look at the pinout image in the [pinout chapter](#)[(#connectors)].



Qwiic OLED display

- 1 Connect the OLED display to one of the Qwiic connectors, you can use either the left one or the right one



Qwiic OLED display

- 1 We've prepared the example code and the libraries in [this](#) zip file.
- 2 Extract the files and open the folder
- 3 Install mpremote

mpremote is a Python module needed to upload files on the Nano ESP32. The minimum suggested mpremote release is 1.22.0. Be sure to have Python installed before proceeding!

```
1 (venv)$ pip install mpremote
```

or

```
1 (venv)$ python3 -m pip install m
```

Depending on how you configure Python on your machine.

- 1 Install library

Run the following line to upload all files and download the dependencies needed to run the Arduino Alvik MicroPython library.

Linux

Windows

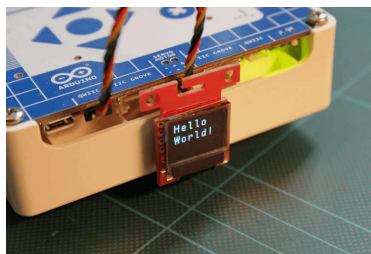
```
1 > install_oled_lib.bat -p <device port>
```

The `<device port>` is the name of the USB port that your computer assigned to the Nano ESP32. There are several ways to find it, depending on your Operating System, for example:

You can use the Arduino IDE to know the port by [following this guide](#). You can look it using the Arduino Lab for MicroPython by clicking `Connect` after have connected the Alvik with the USB cable. You can look at the list of the USB devices attached to the PC

1 Test `Hello World!`

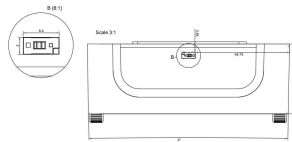
Now you can open the Arduino Lab for MicroPython, connect Alvik and open the example called `hello_world.py` in the `examples` folder. If everything works as expected you'll see something like the following image:



Hello World OLED

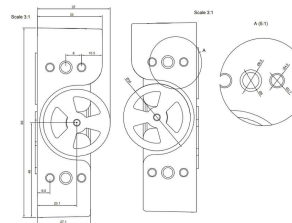
Mechanical Information

Understanding Alvik's dimensions and mechanical details is key for designing custom components and ensuring it works well with other hardware. This information is here to help you easily create and adapt Alvik for your specific projects and needs.



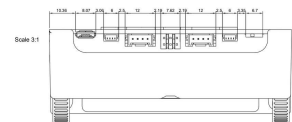
Alvik's Front Side Dimensions

Side



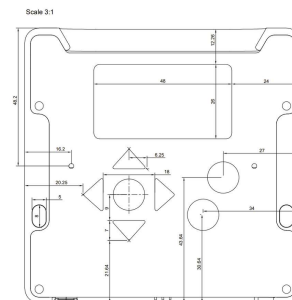
Alvik's Side Dimensions

Back



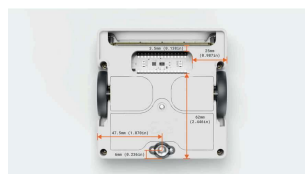
Alvik's Back Side Dimensions

Top



Alvik's Top Side Dimensions

Bottom



Alvik's bottom side dimensions

Want More?

If you're looking to get the most out of your Alvik robot, we have a comprehensive

projects and lessons designed to help you master your Alvik. This course provides step-by-step instructions and detailed explanations to enhance your robotics and programming skills. Whether you're a beginner or an experienced maker, our course will take your Alvik experience to the next level.

Explore our hands-on, step-by-step solutions that support students in their first steps in programming, electronics, and science. Linked directly into the curriculum, no prior knowledge or experience is required.

Need Help

If you need further assistance, you can find support through the following channels:

[Arduino Discord](#)

[Arduino Forum](#)

[Arduino Contact Us Form](#)

For more detailed support, you can also reach out to the Arduino Help Center.

Suggest changes

The content on [docs.arduino.cc](#) is facilitated through a public [GitHub repository](#). If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino](#)
[Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

